Introduction to Architecture Exam

ExamTag: 2023-Jun

Instructions

Format: This exam consists of 6 multiple choice questions for each of the 12 lectures in the course. You may have to do some calculations in order to determine the correct answer. The motivation for having so many questions is to get a more accurate assessment of your knowledge of the material.

Be sure to circle the answers on the page titled "answers page" or your answers will not be graded. If you choose not to answer a question do not circle a letter. Feel free to detach the answers page from the remainder of the exam and only turn it in.

Grading: To discourage guessing, each incorrect answer will be worth -1/3 point, while each correct answer is worth +1 point. If you don't answer you will get 0 points for that question. I.e., if you guess randomly, the expected score is 0.

Material: You are allowed a calculator and one double-sided, hand-written A4 sheet of notes during the exam.

Good luck!

Note: Please take this exam seriously and pass it! (Exams take huge amounts of teaching time to produce and grade, and most students and teachers would prefer that the time be spent teaching instead.)

1. ISA 1

1. How do we check that the address is correctly word aligned in our MIPS architecture?
    a. Address is a multiple of 2
    b. Address is a multiple of 4
    c. Address has its last two bits both 11
    d. Address does not point to a byte

2. Whenever the operating system switches between programs it has to store all of the current program's state (all the data stored inside the processor that the program is using) so that when it switches back it can restore it to where the program was with the data it needs. How many bytes are required to store the state we have used in our MIPS programs?
    a. 31
    b. 32
    c. 124
    d. 128

3. What is the value of R2 after this MIPS code is executed? (ignore branch delay slots)

```
R2 = 4, R12=8
beq     R12, 17, skip
addi    R2, R0, 18
j       done
skip:
        addi    R2, R0, 20
done:
```

    a. 18
    b. 20
    c. 22
    d. Program won't work as intended (Assembly is incorrect)

4. Why do we need both a register read and an ALU operation to write to memory?
    a. We don't
    b. The ALU always does something regardless of whether we use the results
    c. The register provides the write data and the ALU calculates the address
    d. The register provides the address and the ALU proves the write data

5. Which of the following will NOT copy the contents of R2 to R1?
    a. addi R1, R2, 0
    b. sll R1, R2, 0
    c. sw R1, R0(12)
       lw R2, R0(12)
    d. add R1, R2, R0

6. How much space does a branch label take up in the machine code?
    a. 0 words
    b. 1 word
    c. 1 byte
    d. 1 byte per character in the label name

2. ISA 2

1. What is the maximum distance (in instructions) that a conditional branch instruction can branch in the MIPS ISA?

Name	Bit Fields					
	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits
R-Format	op	rs	rt	rd	shmt	funct
I-format	op	rs	rt	**address/immediate (16)**		
J-format	op	**target address (26)**				

   a. $[0, 2^{16}]$
   b. $[-2^{15}, 2^{15}-1]$
   c. $[-2^{16}, 2^{16}]$
   d. $[-2^{16}, 2^{16}-1]$

2. A designer decides to add a fused multiply-add (FMA) instruction to our MIPS processor. The instruction does the following operation on registers: A=A*B+C. What type of instruction format can we use to encode this new instruction? (Assume Register File is extended to allow three concurrent reads)

   a. R
   b. I
   c. J
   d. Need a new format

3. Why is it good to have consistent instruction encoding (MIPS) as opposed to variable or complex instruction encoding (x86)?

   a. Smaller instructions
   b. Easier to handle constants
   c. Easier to decode instructions
   d. All of the above

4. What registers does this method need to save and restore when it starts and returns to follow the MIPS calling convention? Hint: `$t*` registers are caller saved, `$s#`, `$sp`, `$fp`, `$ra` registers are callee saved.

```
method3:
        addi    $t0, $zero, 0
        beq     $a0, $zero, end
        sll     $t0, $a0, 2
        add     $t0, $t0, $a1
end:
        addi    $vo, $t0, 0
        jr      $ra
```

   a. None
   b. $t0
   c. $t0, $a0, $a1
   d. $t0, $a0, $a1, $ra

5. How long will this program take to get to finish? (Assume each instruction takes 1 cycle; e.g., no delay slots.)

```
start:
  jal middle
  finish:
middle:
  jal last
  jr $ra
last:
  jr $ra
```

   a. 3 cycles
   b. 4 cycles
   c. 8 cycles
   d. Forever

6. To make procedure calls faster, a designer decides to change the JAL instruction to increment the stack pointer by 2 words automatically and JR to decrement the stack pointer by 2 words automatically. The designer adds special hardware to the processor to do this in the same time as the regular JAL and JR instructions. What effect does this have?

   a. Procedures that need to save 1 or 2 registers are faster
   b. Procedures that need to save more than 2 registers are not faster
   c. Procedures that save fewer than 2 registers waste space on the stack
   d. All of the above

3. Computer Arithmetic

1. What are the steps required to compute `A - B` with two's complement.
    a. Invert A, add A and B
    b. Invert A, add A and B and 1
    c. Invert B, add A and B
    d. Invert B, add A and B and 1

2. What is the result of the following signed-magnitude addition: 1000+0001?
    a. -1
    b. 0
    c. 1
    d. 8

3. For a 7-bit normalized floating-point format: $(-1)^S*(1.FFFF)*(2^{EE})$ where FFFF is unsigned and EE is two's complement, what is the smallest positive value you can represent?
    a. 0.5
    b. 0.25
    c. 0.125
    d. 0.28125

4. The number 0110 cannot represent which of the following in fixed point:
    a. 0.375
    b. 0.625
    c. 0.750
    d. 3.000

5. A serial multiplier trades off what for what vs. a parallel multiplier? (The serial multiplier is...)
    a. Lower precision, but faster
    b. Faster, but larger circuit area
    c. Slower, but smaller circuit area
    d. They are the same, just different ways to look at the circuit

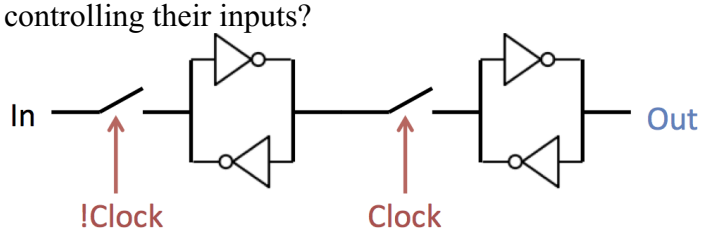
6. You want to compute a two's complement add, but accidentally use an unsigned add instruction instead. What can you say about the result, compare to a signed add?
    a. Signed and unsigned add is the same in hardware, result is always the same
    b. The output is the same only if the input values are positive
    c. The output is always the same, but the overflow flag may be incorrect
    d. It reduces the instruction latency

4. Logic

1. What does a latch do?
    a. Store a value
    b. Update the stored value on the clock signal
    c. Break the feedback loop in a sequential circuit
    d. All of the above

2. As circuits get smaller, the size of the DRAM cells is shrinking. This means there are fewer and fewer electrons to store each bit in the DRAM. What implications does this trend have?
    a. The DRAM will become more reliable since it is easier to know whether you read out a 1 or 0 if there are fewer electrons.
    b. The DRAM will become faster since you won't have to wait as long to get all the electrons out to see if it is a 0 or 1.
    c. The DRAM will be more sensitive to cosmic rays that leave random electrons on the chip.
    d. All of the above

3. What is the benefit of having two latches back to back with opposite clock signals controlling their inputs?

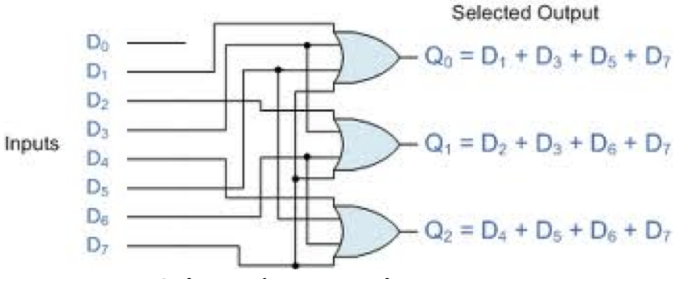


    a. The data is only updated when the clock goes from low to high
    b. The data only goes through the latch when the clock is low
    c. The latch can store a value when the clock is high
    d. The latch behaves the same as a single latch but is slower

4. Which is faster, and why? DRAM or SRAM?
    a. DRAM: each bit is stored in a powered feedback loop so it can be read quickly
    b. DRAM: each bit is stored in a passive capacitor so it can be read quickly
    c. SRAM: each bit is stored in a powered feedback loop so it can be read quickly
    d. SRAM: each bit is stored in a passive capacitor so it can be read quickly

5. A designer connects a long chain of AND gates together. (E.g., the output from the first AND goes into one of the inputs of the second AND and so forth.) The circuit uses 1.0V to represent 1 and 0.0V to represent 0. When the circuit comes near a mobile phone it experiences +0.1V of noise on the wires between each AND gate. How many AND gates can be put in a chain until the noise can cause the wrong result?
    a. 0
    b. 1
    c. 5
    d. No limit

6. Why is the D0 input to an encoder not connected to the logic that determines the output?



a. D0 is a dummy input
b. If D0 is true, then we want the outputs to be all false
c. D0 is an illegal input
d. The logic doesn't need D0 as an input because one of the outputs is always true

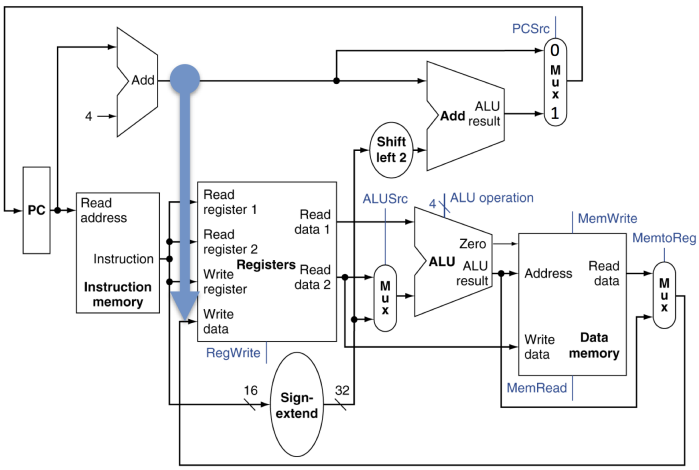
5. Processor Control and Datapath

1. What is **not** true about Instruction Fetching (IF) in the Datapath?
    a. The IF keeps track of the current instruction address using the PC register
    b. The instruction pointed to by the PC is loaded from the instruction memory
    c. Once the instruction is loaded from instruction memory, it is stored in the register file to be decoded.
    d. The PC+4 value is calculated every cycle

2. A single-cycle processor has three types of instructions: A takes 1ns, B takes 2ns, and C takes 10ns. A program with an instruction mix of 25% A, 70% B, and 5% C will run at what clock speed?
    a. 1000 MHz (1/1ns)
    b. 465 MHz (1/2.15ns)
    c. 333 MHz (1/3ns)
    d. 100 MHz (1/10ns)

3. Why did our single-cycle processor design have two memories?
    a. It doesn't, we just treat them as two different memories to understand better
    b. Different parts of the instruction access the instruction memory and the data memory
    c. Some instructions need to access the instruction memory and the data memory at the same time
    d. We don't have a cache so it is faster if we have two memories

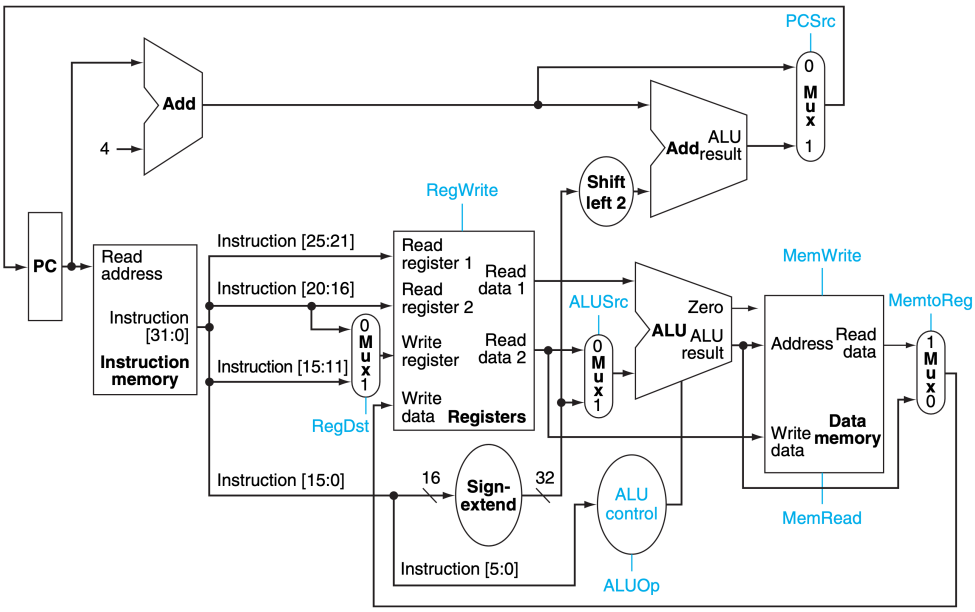
4. Which instructions need the connection shown in the datapath below with the thick line?



    a. jr
    b. jal
    c. jr and jal
    d. jr, jal, and j

5. Why do we not have hazards in the single-cycle processor?
    a. Every instruction finishes before the next one starts so the register file always has the right data
    b. No instructions are executing in parallel so there are no conflicts between instructions
    c. Each instruction has access to the whole processor while it is executing so there is no fighting for resources
    d. All of the above

6. What is read out of Read data 2 in the register file for an I-format instruction?



Name	Bit Fields					
	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits
R-Format	op	rs	rt	rd	shmt	funct
I-format	op	rs	rt	**address/immediate (16)**		
J-format	op	**target address (26)**				

a. Nothing
b. The immediate
c. The value stored in the register identified by the top 5 bits of the immediate value
d. For immediate ALU instructions (e.g. addi, etc.) the value stored in the register that this instruction will be writing to

6. Pipelining

1. What is the purpose of pipelining a processor?
    a. To decrease the instruction latency
    b. To lower the clock frequency by split up stages
    c. To utilize different parts of the processor at the same time and do meaningful work
    d. To increase the effective capacity of memory

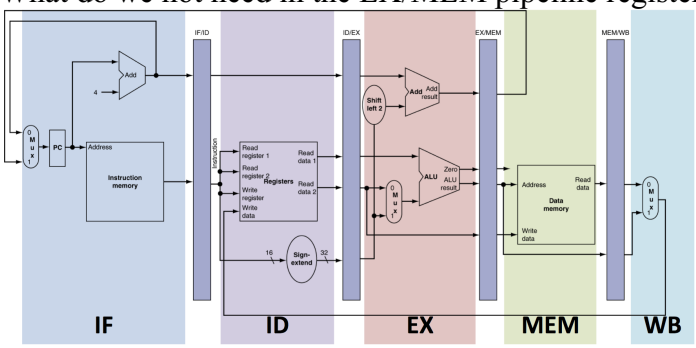
2. What does the pipeline do that causes problems with the ISA's promise of atomic execution?
    a. Nothing: there is no problem
    b. It executes parts of instructions across multiple stages
    c. It executes multiple instructions at the same time
    d. B and C

3. Why do single-cycle processors have slow clock speeds?
    a. They have to handle a different length instructions
    b. They are limited by the slowest instruction
    c. They can't do any instructions in parallel
    d. All of the above

4. What is the most important thing for getting performance from a pipelined design?
    a. Have as few pipeline registers as possible
    b. Make each stage fast
    c. Balance the latency of each stage
    d. Keep the pipeline full

5. A processor is perfectly pipelined with 5 stages and runs at 5x the clock speed with no overhead, but the pipeline is only 80% full. What is the speedup over the original processor?
    a. 0.8x
    b. 4x
    c. 5x
    d. 20x

6. What do we not need in the EX/MEM pipeline register?



    a. ALU result
    b. Register read data 1
    c. Register read data 2
    d. Branch address

7. Hazards

1. Given the following code, what is **not** correct about the instructions and hazards? Assume no double pumped registers and no data forwarding.
```
Start:
1: addi $t0, $r0, 30
2: addi $t1, $r0, 12
3: bne $t0, $t1, Start
4: add $t2, $t0, $t1
5: addi $t0, $t2, 10
```
   a. Line 1 and 4 cause a structural hazard
   b. Line 3 causes a control hazards
   c. Line 1 and 3 cause a data hazard
   d. Line 3 and 4 cause a data hazard

2. How can forwarding help alleviate structural hazards?
   a. By forwarding a value to the previous stage
   b. By forwarding a value to the next stage
   c. A and B
   d. Forwarding cannot alleviate structural hazards

3. What is needed to resolve any hazards in this example?
```
addi R14, R14, 8
lw R1, 4(R13)
add R1, R1, R13
sw R14, 4(R13)
```
   a. Forwarding from MEM to EX
   b. Forwarding from WB to MEM
   c. Can't resolve; need to stall
   d. There are no hazards to resolve

4. What is needed to resolve the following hazard?
```
xor R12, R13, R12
ld R2, R3
add R13, R4, R12
```
   a. Double-pumping the register file
   b. Forwarding from WB to EX
   c. Forwarding from MEM to EX
   d. There is no hazard to resolve, there are enough instructions between the dependencies.

5. In a certain program run on a 5-stage pipeline processor with double-pumped register file but no forwarding, 50% of all instructions are loads and 10% of the instructions directly following the loads depend on it. What is the slowdown of the program due to these dependencies?
   a. 1.0x (no slowdown)
   b. 1.05x
   c. 1.1x
   d. 1.2x

6. Are branch delay slots a good idea?

    a. Yes: the compiler can put useful instructions in them almost all the time
    b. Yes: people typically recompile their programs for new processors
    c. A and B
    d. No: all future processors have to emulate them to maintain compatibility

8. Branch Prediction and Exceptions and Interrupts

1. What is correct about dynamic branch predictors?
    a. Dynamic branch predictors learn branch patterns while running the program.
    b. Current systems do not use dynamic predictors but use static predictors instead
    c. A dynamic branch predictor always requires one bit per branch entry
    d. Dynamic branch predictors always perform better than static branch predictors

2. What is prediction accuracy of a 2-bit branch predictor when running the following code?
```
for (i = 0; i < 1000; i++) // Loop 1000 times {
  if (i % 2 == 0) { //If value of i is even
    //Do something
  }
}
```
Assume that the branch predictor begins with an initial state of strongly taken (11) and that the branch predictor can track multiple branches.
Branch 1: `for` loop, Branch 2: `if` statement.
    a. Branch 1: 997/1000, Branch 2: 500/1000
    b. Branch 1: 998/1000, Branch 2: 750/1000
    c. Branch 1: 998/1000, Branch 2: 500/1000
    d. Branch 1: 999/1000, Branch 2: 500/1000

3. For a given program 80% of the branches are taken and our branch predictor is 75% accurate. A correct branch prediction costs 1 cycle and a misprediction costs 8 cycles. 12% of the executed instructions are branches. How much slower would the processor run (compared to a proeccor with a 100% accurate branch predictor). Assume that all other non-branch instructions have a CPI or IPC of 1.
    a. 0.80x
    b. 0.88x
    c. 1.21x
    d. 1.84x

4. Why do we predict branches in modern processors?
    a. Can't afford the extra transistors to calculate the branch earlier
    b. The pipelines are long and we can't move the branch calculation earlier
    c. Branch predictors are faster than calculating the branch
    d. Predicting a branch is nearly perfectly accurate

5. When an exception occurs, what happens to the other instructions in the pipeline?
    a. They are all killed
    b. They are killed if they come after the instruction with the exception
    c. They are killed if they have a data dependency on the instruction with the exception
    d. The operating system decides which instructions to kill

6. If we add a branch target buffer (BTB) to the branch predictor, do we still need an adder to calculate the branch address?
    a. Yes, but only if we predict the branch incorrectly
    b. Yes, to verify that the target was correct even if we predict correctly
    c. Only the first time we see the branch so we can store the address in the BTB
    d. No, the instruction provides the address

9. Input/Output

1. What are the benefit(s) of polling over interrupt-based I/O?
    a. Polling can detect ready data sooner
    b. Polling uses less processor time
    c. Polling has higher throughput for large data transfers
    d. All of the above
2. Why don't we use DRAM for data storage instead of disks and flash?
    a. Too expensive
    b. Too slow
    c. Too unreliable
    d. Something else
3. How do DMA and memory-mapping interact?
    a. DMA cannot be used if the device is memory-mapped
    b. DMA uses the device's interrupt to move data
    c. DMA moves data by accessing the device directly
    d. DMA transfer is set-up using memory-mapped I/O
4. Why do serial communications send two versions of the same signal?
    a. To check against each other to make sure the final signal is correct
    b. To subtract from each other to cancel out noise that affects both signals
    c. To reduce energy by sharing noise between the wires
    d. All of the above
5. Rank the following technologies in terms of cost per MB: flash, DRAM, hard disk, tape
    a. (cheapest) flash < DRAM < hard disk < tape
    b. (cheapest) tape < hard disk < DRAM < flash
    c. (cheapest) tape < DRAM < flash < hard disk
    d. (cheapest) tape < hard disk < flash < DRAM
6. What is true about the following?
    a. Recently spinning HDD performance has improved greatly
    b. The performance of SSDs has forced the industry to introduce a new I/O protocol.
    c. As the speed of disks increased, the relative software overhead became smaller.
    d. With faster disks, it is more favorable to use interrupt, not polling.

10. Caches

1. What is true about dirty bits in caches?
    a. Cache lines with dirty bits need to eventually be written back to the main memory (DRAM)
    b. Dirty bits are required for write-through caches
    c. A dirty bit shows that the data has changed in the memory (DRAM), but not in the cache yet.
    d. Bits loose voltage over time which can corrupt data. Dirty bits are set for such data that have been corrupted/dirtied

2. Which of the following types of caches does not need a replacement policy?
    a. Direct-mapped caches
    b. Set-associative caches
    c. Fully-associative caches
    d. All types of caches need replacement policies

3. What is true about large cache blocks (or cache lines)?
    a. Bigger blocks result in more bits used in the tags (wasting space)
    b. Bigger blocks are not effective for sequential data accesses
    c. Bigger blocks provide higher spatial locality
    d. Real processors tend to have very small cache blocks (on or two words)

4. What is **not** true about the following:
    a. Tag bits in the address are used to verify whether the data in the cache is indeed the data that we want to access/modify.
    b. Index bits in the address are used to locate which way of the cache to lookup.
    c. Index bits are not required for fully-associative caches
    d. Fully-associative caches require more HW logic than direct-mapped caches.

5. Assume a 4-entry fully-associative caache with one-byte per line, and a LRU replacement policy. How many misses occur for the following accesses?
   `0, 1, 2, 3, 4, 5, 3, 2, 1, 2, 5`
    a. 6
    b. 7
    c. 8
    d. 9

6. Why do we have a memory hierarchy?
    a. To make the memory look big
    b. To make the memory look fast
    c. Because we can't afford tons of SRAM
    d. All of the above

11. Virtual Memory

1. Writing to which address with the following page table would result in a page being loaded from disk?

```
VA-->PA          on disk          access bits
2-->8            0                read/copy on write
1-->9            0                read/write
14-->7           0                read/write
16-->14          1                read/write
13-->6           1                read only
```

   a. 8
   b. 13
   c. 14
   d. 16

2. Two programs want to share data. Can this be done with virtual memory?
   a. No, each program has its own virtual address space that maps to its own physical memory.
   b. Yes, they can both have pages that map to the same physical memory.
   c. Only if they use a physically-index and physically-tagged cache.
   d. Only if they use a virtually-index and virtually-tagged cache.

3. What are the benefits of virtual memory?
   a. It increases security by isolating each process' memory
   b. It frees the applications from having to manage a shared memory
   c. It allows an application to use more memory than physically available
   d. All of the above

4. Computers typically use 4kB pages, but often support larger (2MB and 1GB) pages as well. What is the advantage of larger pages for programs that access data sequentially?
   a. Fewer TLB misses
   b. More TLB misses but faster DRAM access
   c. Can access larger regions of memory in the program
   d. All of the above

5. Why is a Virtually Indexed, Physically Tagged cache a good design?
   a. You can access the TLB and the cache tags in parallel to save time.
   b. The cache index address is unique across programs to provide security.
   c. Using separate types of addresses allows the TLB and cache to be smaller so they are faster.
   d. All of the above.

6. Why do we have multi-level page tables?
   a. To reduce the maximum size of the page table
   b. To reduce the in-memory size of the page table
   c. To make the TLB faster
   d. All of the above

12. Parallelism

1. Why do we do parallelism?
    a. Faster
    b. Easier
    c. Lower power
    d. All of the above

2. How do locks prevent problems with multiple processors accessing data at the same time?
    a. They tell the hardware to prevent multiple accesses to the data
    b. They tell the hardware to keep the data synchronized across multiple processors
    c. They tell the software when it's processor is allowed to access the data
    d. They allow the software to coordinate which processor accesses the data

3. What happened in 2003/2004 that led us to have multicore chips everywhere today?
    a. Processors got small enough that you could put multiple cores on the same chip to get better performance
    b. Processors got so hot that it was too expensive to cool them, so manufacturers switched to putting multiple slower (cooler) processors on a chip
    c. Manufacturers couldn't increase the clock speed, so they put multiple cores on the same chip to get better throughput
    d. All of the above

4. How much faster can my program run if I have 9000 cores and 10% of my program cannot be parallelized?
    a. 9000x faster
    b. 4500x faster
    c. 5x faster
    d. 10x faster

5. What do you **not** need to add to the pipeline to support executing load/store instructions in parallel with other (non-load/store) instructions?
    a. Extra ALU
    b. Extra data memory ports
    c. Extra register file ports
    d. Extra instruction memory ports

6. Which of the following processor designs can execute more than one instruction at the same time?
    a. Pipelined Single-issue
    b. Pipelined Dual-issue
    c. Pipelined Symmetric Multithreading (SMT)
    d. All of the above

Make sure you circle the answers on the answer page.
Answers on other pages will NOT be graded.